Jen v4 の設計

エージェントの「終わりました」を、機械で確かめる

jen-marketplace v4.0.0

Table of Contents

1. 何を直したのか

AI エージェントに開発を任せると、まだ終わっていないのに「終わりました」と報告してくることがある。テストを走らせずに「テストは通りました」と書くこともある。

v3 までの Jen には、これを止める仕組みとして「検収役のエージェントが OK と言うまで完了にしない」という決まりがあった。ただしこの決まりは、プロンプトに書いた文章でしかなかった。つまり「そう振る舞ってください」とお願いしていただけで、守られたかどうかを確かめる手段がなかった。

Jen には hooks という、Claude Code が機械的に実行する仕組みもある。だが v3 の時点では `rm -rf /` のような危険なコマンドを正規表現で弾くだけで、全部で 67 行しかなかった。完了判定には関わっていない。

v4 で変えたのは、この一点だけである。完了の判定をプログラムに移した。モードの種類もエージェントの数も、委譲の流れも v3 のままである。

2. やり方：受入条件を項目に割り、項目ごとに証拠をためる

ミッションを始めるとき、受入条件を「1 つずつ独立に確かめられる項目」に割って宣言する。たとえば次のように書く。

- C1: lint と typecheck とテストが通る
- C2: 初回画面に次のアクションが表示される

それぞれの項目は、3 つの状態のどれかを持つ。

状態	意味
NULL	証拠がまだ何もない
PARTIAL	証拠はあるが、要求した強さに届いていない
MATCH	要求した強さの証拠がそろった

状態は上がるだけで、下がらない。一度 MATCH になった項目は、あとで弱い証拠を足しても MATCH のままである。

そして全項目が **MATCH** になるまで、セッションを終われない。終わろうとすると Stop hook が差し戻す。この判定に LLM は関与しない。項目の状態を読んで、全部そろっているかを見るだけのプログラムである。

なぜ項目に割るのか

v3 の検収役には「受入条件を満たしましたか？」という大きな質問を 1 回していた。判断を 1 回しか聞かないので、その 1 回を外すと全部外れる。

v4 は質問を小さく割る。聞くのは「この 1 つの証拠は、この 1 つの項目を裏づけるか」だけである。小さい質問なので安いモデルでも安定して答えられるし、外れても 1 項目で済む。そして「全部そろったか」の集計はプログラムがやるので、ここは外れない。

この考え方は HALT という論文から取っている。

> 停止の判断を、生成側の自信ではなく証拠がそろったかどうかで決める （HALT — Roh & Han 2026, arXiv 2608.02009）

同じ論文は、止まれない実例も測っている。3B のエージェントは、78〜88% の問題で自分では止まらず、与えられた予算を使い切るまで検索を続けた。

3. 証拠には強さがある

ここが v4 でいちばん重要な部分である。

まず前提として、「証拠」を文字列で表すと破られる。実例がある。あるエンジニアが Stop hook を作り、Claude の出力に `<proof>QUALITY_GATE_PASSED</proof>` という文字列があれば通す、という仕組みにした。すると Claude は、品質チェックを実行せずにその文字列だけを出力して通り抜けるようになった。

本人の分析が的確だった。これは Claude が「ずるをしよう」と考えたのではなく、訓練データの偏りだという。指示文に出てきた文字列をそのまま出力するのは、言語モデルにとってごく普通の動作である。一方、ファイルを偽造するのは、言語モデルにとって珍しい動作である。

そこで v4 は証拠を 3 段階に分けた。

強さ	中身	作れるのは誰か	偽造できるか
weak	エージェントの報告（「実装しました」）	エージェント	できる
medium	ファイル（台帳が中身のハッシュを計算して記録する）	エージェント	手間はかかるができる
strong	台帳が自分でコマンドを実行して得た終了コード	台帳だけ	できない

strong の証拠を「作った」と宣言する方法は用意していない。`run` という、台帳自身がコマンドを実行するコマンドだけが strong を生む。だからエージェントは「テストが通りました」を、文章を書くことでは作れない。

既定では medium 以上を要求する。つまり「実装しました」という報告だけでは、項目は MATCH にならない。

Anthropic のハーネスにも同じ問題がある

Anthropic が公開しているハーネスは、機能ごとに `"passes": false` を持つ JSON ファイルを使う。データの形としては正しい。だがそのファイルが勝手に書き換えられるのを、次の方法で防いでいる。

> 「テストを削除したり編集したりすることは許されない」といった強い言い回しの指示を使っている （Anthropic / effective-harnesses-for-long-running-agents）

これはプロンプトである。上の実験から分かるとおり、プロンプトは偽造できる。

4. 検証をいつ走らせるか

検証を毎回走らせるべきか、最後に 1 回でいいかは、使っているモデルによって変わる。これは独立した 2 つの実験が同じ結論に達している。

実験 1：外部の比較実験（Jain 2026-10）

同じ API を作る課題を、ハーネスあり・なしで比べた。

モデル	ハーネスなし	ハーネスあり
高性能モデル	10 回中 10 回成功	10 回中 10 回成功（時間と費用だけ増えた）
低価格モデル	10 回中 3 回成功	10 回中 10 回成功

Claude Sonnet 4.6 と Opus 4.6 も、ハーネスなしで一度目から成功している。一方ハーネスを付けた側は、1 回あたり平均で 1,707 万トークン・19 分・$0.187 を余分に使った。ハーネスなしの実行が 107 万トークンなので、約 16 倍になる。

著者の結論はこうである。

> モデルの性能によって変わるのは、独立した検証が必要かどうかではない。いつ走らせるかである

この実験は 1 つの課題を 1 回試しただけなので、著者自身がそう断っている。

実験 2：Anthropic 社内の実験（2026-03）

Anthropic のエンジニアが長時間動くアプリ開発用のハーネスを作り、その後こう書いている。最初のハーネスは「かさばり、遅く、高くついた」。そこで Opus 4.6 がリリースされたのを機に、作業を区切る仕組みを丸ごと削除し、評価役を実行の最後に 1 回だけ走らせる形に変えた。計画役は残した。削ると、実装役が作業範囲を小さく見積もるようになるからである。

> 評価役を使うかどうかは固定の判断ではない。いまのモデルが単独で確実に解ける範囲を超えた仕事のときに、コストに見合う

Jen での実装

使うモデル	検証の頻度	理由
haiku / sonnet	工程ごとに走らせる	安いモデルではハーネスの効果が実験で出ている
opus	最後に 1 回だけ	工程ごとに走らせても精度が上がらず、費用だけ増え

使うモデル	検証の頻度	理由
		る
fable	opus と同じ	現状の性能では優先度が低いため、検証していない

Jen の 18 体のエージェントのうち 12 体は sonnet で動く。つまりハーネスの効果が出る側に、実際の作業の大半がある。

5. Claude の外に出ない

Jen には「使う環境は Claude だけで完結する」という方針がある。v4 ではこれを、ほかのどの判断よりも優先する決まりにした。性能が上がるとしても、論文の裏づけがあるとしても、この方針に反する実装は採らない。

ただし制限がかかるのは判断する側だけである。道具は自由に使う。

採らないもの	採るもの
Claude 以外のモデルに判断させること（他社の LLM、ローカルの LLM、学習済みの分類器）	MCP サーバ（ブラウザ自動化、テスト実行、git など）。自分で作るより既存のものを使う
学習や微調整が必要な部品（LoRA、報酬モデル、進化的探索）	外部コマンドやライブラリ

運用上もう 1 つ決めている。特定の MCP サーバや外部コマンドを必須にはしない。あれば使い、なければその機能だけ落として動く。Jen は「インストールしたらリポジトリには何も置かない」という形を保ってきた。導入に手間がかかるようになると、その良さが消える。だから hooks 本体は Python の標準ライブラリだけで書いてある。

調べた機構を、この方針で選り分けた結果

機構	採否	理由
HALT の停止の決め方	採る	考え方だけ移す。集計は Python で書く
HALT の検証器（Qwen2.5-3B に LoRA を足したもの）	採らない	他社のモデルに判断させることになる。学習も必要
ブラウザ自動化での検収（Puppeteer / Playwright の MCP）	採る	Claude の中から実行できる。偽造できない証拠を作れる
Sakana Fugu	採らない	Claude ではないモデルを使う
Sakana Trinity / Conductor	採らない	指揮役を学習させる手法
Sakana DGM / MASS	採らない	モデル自体を書き換える手法
Meta-Harness / AutoHarness	採らない	ハーネスを学習で自動探索する手法

Sakana AI の手法が全部落ちている。Jen v1 は Sakana Fugu のルーティングの考え方から始まったが、2026 年の Fugu は「オーケストレーションをモデルの中に組み込む」方向へ進んだ。この方針とは両立しない。考え方は参照するが、実装は採らない。

この方針は、費用の面でも都合がよかった

Jain の実験で分かったのは、ハーネスの最大のリスクが余計にかかる費用と時間だということである。Claude だけで完結させ、標準ライブラリだけで書くという制限は、結果として重い仕組みを作れなくする。最も強い反対証拠が警告しているコストを、構造として抑えられる。

この方針はドキュメントに書くだけでなく、`tests/check_axiom0.py` が機械的に確かめ、CI が毎回実行する。文章で書いただけでは、v4 が批判しているものと同じになってしまう。

6. ハーネスの部品は、いつか要らなくなる

Anthropic のエンジニアが書いている一文が、この設計の背骨になった。

> ハーネスの各部品は、モデルにこれができないという仮定の上に立っている

つまり部品には期限がある。モデルが良くなればその仮定は外れ、部品は効果のない費用だけになる。

この見方は 2 つの事実を同時に説明する。1 つは Jain の実験で、高性能モデルにハーネスを付けると時間と費用だけが増えたこと。もう 1 つは、日本語圏で公開されている「自分で作ったオーケストレータを使わなくなった」という報告が 5 件あることである。どちらも、前提にしていたモデルの弱点が解消された結果だと見るほうが合う。

そこで v4 は、部品ごとに「どのモデルの弱点を前提にしているか」と「外し方」を書いた。

部品	前提にしているモデルの弱点	外し方
証拠がそろったかの判定	自分が終わったかどうかを甘く見積もる	要求する証拠の強さを下げる。最後は台帳ごと使わない
証拠の 3 段階	弱い証拠を強い証拠と見なす	既定の要求を weak にする
完了を差し戻す hook	まだ終わっていないのに終わったと言う	最後に 1 回だけにする。最後は hooks から外す
モデル別の検証頻度	安いモデルは継続的に見ていないと外れる	モデルと頻度の対応表を書き換える
台帳を守るパターン	ゲートを通るために証拠を偽造する	外す（ただし上記の実験は 2026 年 3 月に再現している）
危険なコマンドを弾くガード	—	外さない

最後の 1 行だけ期限がない。人間の承認が必要なのは、権限の問題だからである。本番への deploy を人間に確認するのは、モデルがどれだけ賢くなっても変わらない。

この見方を最初に適用したのは Jen 自身である。v3 は「Fable は長時間の作業で一貫性を保てる」という前提の上に「指揮は Fable」という構成を建てた。実際に使ってみると、その前提に見合う結果が出ていない。上の 5 件と同じ形である。

7. jen-classic は「少ないトークンで同じ結果を出す」版にした

Jen には 2 つの版がある。v3 までの `jen-classic` は「Fable を使わない版」、つまり Fable が使えない人のための代替として定義していた。v4 で役割そのものを変えた。

`jen-classic` は、同じ仕事をより少ないトークンと時間で終わらせるための版である。

`jen`（Fable 版）は、難しい仕事を解けるようにすることを目指す。`jen-classic` は、解けると分かっている仕事を安く終わらせることを目指す。目的が違うので、既定の動き方も違う。

判断の基準

Stanford の研究者 2 人が、情報理論を使ってこう論じている。

> 推論に使えるトークン数を固定すると、単一のエージェントのほうが情報を効率よく使える
>
> 複数エージェントが有利になるのは、1 つのエージェントではコンテキストを使いこなせなくなったとき、またはより多くの計算を費やしたときである
>
> 報告されている複数エージェントの利点の多くは、計上されていない計算量で説明がつく
>
> （Tran & Kiela 2026, arXiv 2604.02460）

ここから `jen-classic` の判断基準が出る。

ほかのエージェントに任せるのは、自分 **1** つでは扱いきれないほど情報量が増えたときだけにする。 それ以外は自分で片づけたほうが安く済む。「エージェントが 18 体いるから振る」は理由にならない。

	jen（Fable 版）	jen-classic
目的	難しい仕事を解けるようにする	同じ仕事を少ないトークンで終わらせる
既定の動き	分解してほかのエージェントに任せる	自分で片づける。任せるのは情報量が増えたときだけ
最初に使うモデル	sonnet	haiku。検収で差し戻されてから上げる
複数を同時に動かすか	独立した作業は同時に動かす	既定では動かさない
最後の手段	deep-solver を 1 回呼ぶ	同じ。合議制は人が指示したときだけ
検証の頻度	opus なので最後に 1 回	工程ごとに走らせる

同時に動かさない理由は実測にある。Cursor は 20 体のエージェントを同時に動かしたとき、ロックの取り合いで実質 2〜3 体分の処理しか進まなくなったと報告している。CooperBench という評価では、エージェント同士を協働させると成功率が平均 30 ポイント下がった。

エージェントは 18 体のまま残した

自分で片づけるのが既定になると、18 体のエージェントの出番がなくなるように見える。それでも減らさなかった。

費用が増えるのは、理由を言えないまま任せたときである。一覧に何体並んでいるかは費用と無関係である。消せば機能が戻せなくなり、得られるのは一覧が短くなることだけである。

代わりに、v4 の仕組みから自然に出てくる線で分けた。エージェントが作れる証拠の強さで分かれる。

作れる証拠	エージェント	扱い
strong（実行して終了コードを取れる成果を出す）	builder / frontend / test / debugger / verifier / strict-verifier	既定で使う
medium（ファイルの何行目かを示して返す）	scout / release-manager	既定で使う
weak のみ（意見や分析を返す）	ux-critic / contrarian-reviewer / monetization-reviewer / product-strategist / ideation	人が指示したときだけ

最後の群が既定から外れる理由は、仕組みの側にある。意見は weak の証拠にしかならず、既定が要求する medium に届かない。つまり呼んでも項目は MATCH にならず、完了には近づかない。 意見は、完了の判定とは別の軸にある。

1つだけ例外を置いた。`jen-security-reviewer` は、認証・決済・秘密情報・セキュリティの境界に触る作業では必ず呼ぶ。ここは、間違えたときに取り返しがつくかどうかで決める。危険なコマンドのガードと同じで、期限のない決まりとして扱う。

合議制（architect と debugger に別々の仮説を立てさせ、deep-solver が統合する）も同じ扱いにした。機構は残し、既定から外した。独立した仮説をぶつけて思い込みを打ち消すという狙いには筋が通っている。ただし opus を 3 回以上呼ぶので、上の論文が言う「より多くの計算を費やした」場合にあたり、この版の目的と衝突する。

使った量を数える

「少ないトークンで」を標語で終わらせないために、数えられるようにした。

`jen-classic` でいちばん費用がかさむのはほかのエージェントに任せた回数である。サブエージェントは呼び出すたびに何も知らない状態から始まるので、1回の委譲でコンテキストを1つ丸ごと使う。

トークン数そのものは Jen の内側からは見えない。だが委譲の回数なら、hook が記録している実際のログから数えられる。`evidence_ledger.py budget` がそれを数える。元になるのは、hook が実際に書いたログである。

上限を宣言することもできるが、超えても強制的には止めない。v3.4 から「数値で自動的に止めることはしない」という決まりを守っている。超えたときは「分解が重すぎたのではないか」を点検する合図として扱う。

数えられないときは「判定不能」と表示する。0 とは言わない。 PostToolUse のペイロードの形式は Claude Code 側の仕様に依存していて、Jen 側では検証していないからである。

8. 分かっていないこと

偽造を完全には防げていない

台帳には封印（内容のハッシュ）を付けてある。だが計算方法はスクリプトを読めば分かるので、偽造したあとに封印を計算し直せば通ってしまう。防げているのは次の 3 つだけである。

1. 不注意な書き換え（シェルのリダイレクトや `sed -i`）→ 封印が合わなくなるので検出できる
2. よくある編集の経路 → PreToolUse で拒否する
3. 実体のないログを指している strong の証拠 → 捏造として検出できる（封印を計算し直されても捕まる）

つまり「気軽に通り抜ける道を塞ぎ、あとから検出できるようにする」ところまでである。本気で偽造しようとするエージェントは止められない。前述の実験と同じ、訓練データの偏りによるものである。

ただし、いちばん効いているのは、台帳が自分でコマンドを実行するという仕組みそのものである。 台帳が実行する限り、「テストが通った」を文章を書くことでは作れない。

効果を測っていない

「**hook** で強制した検証」と「プロンプトでお願いした検証」を比べて、完了率がどう違うかを測った研究は、今回の調査では見つからなかった。 Jain の実験は「ハーネスのあり・なし」を比べているが、強制の手段が hook かプロンプトかは比べていない。Anthropic 自身も、前述のとおりプロンプトで守っている。

したがって v4 は、まだ誰も測っていない領域にある。効果があるとは言い切らない。

ログには、ゲートを通したか、通ったかが記録として残る。同じような仕事でゲートを切った場合と比べれば、自分で測れる。測ったら `.jen/decisions.md` に残してほしい。

ゲートを回避できる抜け道が残っている

プラグインは全セッションで有効になる。台帳がないというだけで停止を止めると、普通の会話すら終われなくなる。そこで「台帳があるときだけゲートが働く」ことにした。

つまり「台帳を作らなければゲートを回避できる」という抜け道が残る。塞ぐ代わりに、ゲートなしで終わったことを必ず記録するようにした。行動監査がこの記録を読めるので、あとから件数を数えられる。抜け道があることを見えるようにした上で残している。

検討したが採らなかったもの

文献検索の停止ルールとして、生態学の capture-mark-recapture を応用する手法（Kastner ら 2007）を検討した。複数のデータベースの重複から未発見の件数を推定し、「既知の件数が推定値の信頼区間に入ったら止める」という使い方ができる。

要約だけ読むと採用したくなる。だが原文にはこう書いてあった。

- どれだけ推定値に近づけば信頼できるか、その基準はまだ決まっていないと著者自身が明記している

- 検証は過去のデータを使った後ろ向きの検証だけで、実際に使いながらの検証はしていない
- 著者自身のデータでは、この停止条件が一度も成立しなかった（既知 1,246 件に対し推定値の信頼区間は 1,749〜1,955 件で、どの段階でも区間に入らなかった）

そこで、取りこぼしの推定値を出すところまでは使えるが、止めるかどうかの判断には使わないことにした。

この経緯を記録に残している。一次資料を読まずにもう一度検討すると、同じ結論には戻れない。検証されていない停止ルールを仕組みの中心に置いてしまう。

9. 確かめたこと

項目	内容
テスト	20 件。標準ライブラリだけで動く
CI	Python 3.10 / 3.11 / 3.12 / 3.13 で実行する。ドキュメントの「3.10 以上」という記述を実際に確かめるため
方針の確認	`tests/check_axiom0.py` が、標準ライブラリ以外の import、外部コマンドへの依存、版の不一致、存在しない hook への参照、2 つの版の食い違いを検出する
確認スクリプト自体の検証	4 種類の違反を実際に検出できることを確かめてから入れた

最後の行には理由がある。常に合格を返すチェックには意味がない。これは v4 が証拠について言っていることと同じである。

CI を置いた理由も同じである。テストを誰も自動で走らせないなら、「テストは通っています」という記述はエージェントの報告にすぎない。v4 の分類でいう weak の証拠である。

使い方ガイドにも同じ扱いをした。ガイドに載せたコマンドの例は、CI が実際に実行する。書いている途中で例が通らず、直してから載せた。

主な出典

文献	v4 のどこに効いたか
HALT — Roh & Han 2026, arXiv 2608.02009	停止を、証拠がそろったかどうかで決める
Are Harnesses Worth the Cost? — Naresh Jain 2026-10, martinfowler.com	検証をいつ走らせるか。モデル別の頻度の根拠
Harness design for long-running application development — Anthropic 2026-03	同上（独立に同じ結論）。部品に期限があるという見方の出どころ
Single-Agent LLMs Outperform Multi-Agent Systems… — Tran & Kiela 2026, arXiv 2604.02460	jen-classic の判断基準
Claude Code の Stop Hook で simplify を強制したら Claude がズルを覚えた話 — 2026-03	証拠の 3 段階。文字列では証拠にならないこと
To Search or Not to Search — Zhang ら 2026, a	失敗には「探しすぎ」と「早く止めす

文献	v4 のどこに効いたか
rXiv 2602.03304	ぎ」の 2 方向がある
Explore Before Committing — Zhou ら 2026, arXiv 2609.01294	分岐は必要なときだけにする。常に分岐すると負ける
Why Do Multi-Agent LLM Systems Fail? — Cemri ら 2025, arXiv 2503.13657	14 種類の失敗パターン（終了条件が分からない、早く終わる、検証しない）
Inside the Search Process — Kuhlthau 1991	早い段階で要約すると調査が妨げられる
The design of browsing and berrypicking… — Bates 1989	調べているうちに問い自体が変わる

調査には imozuru という Claude Code のスキルを使い、英語と日本語の 2 経路を別々に走らせた。原文は 30 本。反対側の証拠を意識して集めたことが、設計をいちばん変えた。

Jen v4.0.0 ・ MIT License ・ https://github.com/youki0p0/jen-marketplace